

Esquemas Simétricos e Assimétricos: Confidencialidade e Autenticidade

Segurança Informática | ISEL 2025-26

Objetivos de Aprendizagem

- **Compreender** a diferença entre **esquemas simétricos** e **assimétricos**
 - **Identificar** os principais algoritmos e protocolos em cada esquema
 - **Aplicar** conceitos de **confidencialidade** e **autenticidade** em cenários práticos
 - **Analisar** vantagens, desvantagens e casos de uso
-

Introdução à Criptografia

- **Criptografia:** Técnica para proteger informações, garantindo **confidencialidade**, **integridade** e **autenticidade**
 - **Objetivos principais:**
 - **Confidencialidade:** Garantir que apenas destinatários autorizados acedem à informação
 - **Autenticidade:** Verificar a origem e a integridade da mensagem
 - **Classificação:**
 - **Simétrica** (chave única)
 - **Assimétrica** (par de chaves: pública/privada)
-

Esquemas Simétricos

Conceito

- **Chave única** partilhada entre emissor e recetor
- **Vantagens:**
 - **Rápida** (eficiência computacional)
 - **Adequada** para grandes volumes de dados
- **Desvantagens:**
 - **Problema de distribuição de chaves**
 - **Escalabilidade limitada** (número de chaves cresce com o número de utilizadores)

Algoritmos Comuns

- **AES** (Advanced Encryption Standard)
 - **DES** (Data Encryption Standard) e **3DES**
 - **Blowfish**, **RC4**
-

Exemplo Prático: AES (Advanced Encryption Standard)

- **Tipo:** Simétrico (bloco de 128 bits)
- **Tamanhos de chave:** 128, 192 ou 256 bits
- **Modos de operação:**

- **ECB** (Electronic Codebook)
- **CBC** (Cipher Block Chaining)
- **GCM** (Galois/Counter Mode)

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

# Chave (16, 24 ou 32 bytes para AES-128, AES-192, AES-256)
key = get_random_bytes(16)
# IV (Initialization Vector) para modos como CBC
iv = get_random_bytes(16)

# Criar objeto de cifragem (modo CBC)
cipher = AES.new(key, AES.MODE_CBC, iv)
# Dados a cifrar (deve ter comprimento múltiplo de 16 bytes)
data = b"Mensagem secreta!"
padded_data = data + b'\x00' * (16 - len(data) % 16)

# Cifrar
ciphertext = cipher.encrypt(padded_data)
print("Cifrado:", ciphertext)
```

Esquemas Assimétricos

Conceito

- **Par de chaves:**
 - **Chave pública:** Usada para **cifrar** ou **verificar assinaturas**
 - **Chave privada:** Usada para **decifrar** ou **assinaturar**
- **Vantagens:**
 - **Resolve o problema de distribuição de chaves**
 - **Permite assinaturas digitais** (autenticidade e não-repúdio)
- **Desvantagens:**
 - **Lenta** (menos eficiente que a simétrica)
 - **Complexidade computacional**

Algoritmos Comuns

- **RSA** (Rivest-Shamir-Adleman)
- **ECC** (Elliptic Curve Cryptography)
- **Diffie-Hellman** (troca de chaves)

Exemplo Prático: RSA

- **Base matemática:** Fatorização de números primos grandes
- **Processo:**
 1. Gerar par de chaves (pública/privada)

2. Emissor cifra com a **chave pública** do recetor
3. Recetor decifra com a sua **chave privada**

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

# Gerar par de chaves RSA (2048 bits)
key = RSA.generate(2048)
private_key = key.export_key()
public_key = key.publickey().export_key()

# Cifrar com a chave pública
cipher = PKCS1_OAEP.new(RSA.import_key(public_key))
message = b"Mensagem secreta!"
ciphertext = cipher.encrypt(message)

# Decifrar com a chave privada
cipher = PKCS1_OAEP.new(RSA.import_key(private_key))
decrypted = cipher.decrypt(ciphertext)
print("Decifrado:", decrypted.decode())
```

Confidencialidade vs. Autenticidade

Objetivo	Esquema Simétrico	Esquema Assimétrico
Confidencialidade	Cifragem com chave partilhada	Cifragem com chave pública
Autenticidade	MAC (Message Authentication Code)	Assinatura digital (chave privada)

- **MAC:** Usa chave simétrica para garantir integridade e autenticidade (ex: HMAC)
- **Assinatura digital:** Usa chave privada para assinar; qualquer pessoa pode verificar com a chave pública

Comparação: Simétrico vs. Assimétrico

Critério	Simétrico	Assimétrico
Velocidade	Rápido	Lento
Tamanho da chave	Pequena (ex: 128-256 bits)	Grande (ex: 2048-4096 bits)
Distribuição de chaves	Problema (canal seguro necessário)	Fácil (chave pública pode ser partilhada)
Uso típico	Cifragem de dados em massa	Troca de chaves, assinaturas digitais

Casos de Uso Práticos

Simétrico

- Cifragem de **ficheiros** ou **discos** (ex: BitLocker, FileVault)

- Comunicações **TLS/SSL** (usado em conjunto com assimétrico para trocar chaves)

Assimétrico

- **TLS/SSL**: Troca de chaves simétricas via **Diffie-Hellman** ou **RSA**
 - **Assinaturas digitais** em certificados (ex: HTTPS, e-mails assinados)
 - **GPG/PGP**: Cifragem e assinatura de e-mails
-

Boas Práticas

- **Combinações**: Usar **assimétrico** para trocar chaves simétricas (ex: TLS)
 - **Tamanho de chaves**:
 - **Simétrico**: Mínimo **128 bits** (AES-128)
 - **Assimétrico**: Mínimo **2048 bits** (RSA) ou **256 bits** (ECC)
 - **Atualização**: Evitar algoritmos obsoleto (ex: DES, RC4)
 - **Gestão de chaves**: Usar **HSM** (Hardware Security Module) ou **KMS** (Key Management Service)
-

Síntese: Pontos-Chave

- **Simétrico**: **Rápido**, ideal para **dados em massa**, mas requer **distribuição segura de chaves**
 - **Assimétrico**: **Seguro para trocas de chaves e assinaturas**, mas **lento** para grandes volumes
 - **Confidencialidade**: Alcançada com **cifragem** (simétrica ou assimétrica)
 - **Autenticidade**: Alcançada com **MAC** (simétrico) ou **assinaturas digitais** (assimétrico)
 - **Híbrido**: Combinação de ambos (ex: **TLS**) é a abordagem mais comum na prática
-

Referências

- Gollmann, D. (2011). *Computer Security* (3rd ed.). Wiley. ISBN 9780470741153
- Du, W. (2017). *Computer Security: A Hands-on Approach*. CreateSpace. ISBN 9781548367947
- *[FONTE EXTERNA]*: Exemplos de código baseados em bibliotecas padrão (PyCryptodome)